PC 505 AI — Machine Learning

Assignment 2

Complete Answers with Solved Problems

Unit 2 | Unit 3 | Unit 4 | Unit 5

Perceptron · SVM · Bayesian Models · CNN · LDA · PCA · Clustering · RL

Section	Topics Covered
Unit 2	Perceptron, Activation Functions, MLP, Forward Propagation, Loss Functions
Unit 3	SVM, Kernels, Bayesian Models, Naive Bayes, Overfitting, HMM, Bayes Theorem
Unit 4	Linear Regression (Solved), Neural Networks, CNN, LeNet-5, Regularization
Unit 5	LDA (2 Solved Problems), Reinforcement Learning, PCA, K-Means, Clustering

Unit 2 — Perceptron, MLP, Forward Propagation

Activation Functions | Perceptron Limitations | MLP Layers | Loss Functions

Answer

Q1. Describe Perceptron and any two activation functions. What are the limitations of Perceptron?

Perceptron is the simplest artificial neural network unit — a mathematical model of a biological neuron. It takes multiple inputs, computes their weighted sum, adds a bias, and passes the result through an activation function to produce an output.

```
Output = f(w1*x1 + w2*x2 + ... + wn*xn + b) = f(W^T * X + b)
```

Components:

- Inputs (x1,x2,...,xn): feature values fed into the neuron
- Weights (w1,w2,...,wn): importance of each input (learned during training)
- Bias (b): shifts the decision boundary — allows the model to fit data that does not pass through the origin
- Activation function f: introduces non-linearity and decides the output

Activation Function 1 — Sigmoid (Logistic):

```
sigma(x) = 1 / (1 + e^(-x)) Output range: (0, 1)
```

- Smooth, differentiable — good for output layer in binary classification
- Derivative: sigma'(x) = sigma(x) * (1 - sigma(x)) — used in backpropagation
- Limitation: Vanishing gradient problem for very large or small inputs (saturates near 0 or 1)

Activation Function 2 — ReLU (Rectified Linear Unit):

```
ReLU(x) = max(0, x) Output range: [0, infinity)
```

- Most widely used in hidden layers of deep networks
- Computationally efficient — no exponential computation needed
- Solves vanishing gradient for positive values
- Limitation: "Dying ReLU" — neurons output 0 permanently for negative inputs

Limitations of Perceptron:

- Can only classify linearly separable data — fails on XOR problem
- Single layer — cannot learn complex, hierarchical features
- Binary output only — not suitable for multi-class or probabilistic outputs
- No hidden layers — cannot model non-linear relationships

> Classic failure: XOR gate (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0 No single straight line can separate the two classes. Solution: Use Multilayer Perceptron (MLP) with hidden layers.

Answer

Q2. Briefly describe the various layers of Multilayer Perceptron (MLP).

An **MLP (Multilayer Perceptron)** consists of multiple layers of interconnected neurons arranged in a feedforward manner. Each layer performs a specific role:

Layer 1 — Input Layer:

- Receives raw feature data — one neuron per input feature
- No computation is performed here — only passes data to next layer
- Example: For a 28x28 image, input layer has 784 neurons

Layer 2 — Hidden Layer(s):

- One or more layers between input and output — the "learning" happens here
- Each neuron computes: z = W*x + b, output = f(z) where f is an activation function
- Multiple hidden layers allow learning of complex hierarchical patterns
- More hidden layers = deeper network = can model more complex functions

Layer 3 — Output Layer:

- Produces the final prediction of the network
- Number of neurons = number of classes (for classification) or 1 (for regression)
- Activation: Sigmoid (binary classification), Softmax (multi-class), Linear (regression)

> MLP for digit recognition (0-9): Input layer: 784 neurons (28x28 pixels) Hidden layer 1: 256 neurons, ReLU activation Hidden layer 2: 128 neurons, ReLU activation Output layer: 10 neurons (one per digit), Softmax activation Data flows: Input -> H1 -> H2 -> Output -> Prediction

---

Answer

Q3. Describe the Forward Propagation and Loss Functions in MLP.

Forward Propagation is the process of computing the output of an MLP by passing input data layer by layer from input to output. It is the prediction step.

Step-by-step Forward Propagation:

- Step 1 — Input: Feed the input vector X = [x1, x2, ..., xn] into the network
- Step 2 — Hidden layer pre-activation: z(1) = W(1) * X + b(1)
- Step 3 — Hidden layer activation: a(1) = f(z(1)) [apply ReLU, sigmoid, etc.]
- Step 4 — Repeat for each subsequent hidden layer
- Step 5 — Output layer: z_out = W_out * a_last + b_out
- Step 6 — Final output: y_hat = f_out(z_out) [sigmoid/softmax]

```
General: a[L] = f( W[L] * a[L-1] + b[L] ) for layer L
```

Loss Functions measure how far the predicted output is from the actual target. The network minimizes this during training.

1. Mean Squared Error (MSE) — for Regression:

```
MSE = (1/n) * SUM[ (yi - y_hat_i)^2 ]
```

- Penalizes large errors more (squared term) — sensitive to outliers

2. Binary Cross-Entropy — for Binary Classification:

```
Loss = -(1/n) * SUM[ yi*log(y_hat_i) + (1-yi)*log(1-y_hat_i) ]
```

- Measures the difference between two probability distributions
- Output layer uses Sigmoid activation with this loss

3. Categorical Cross-Entropy — for Multi-class Classification:

```
Loss = -(1/n) * SUM_i SUM_k [ y_ik * log(y_hat_ik) ]
```

- Used with Softmax output layer for multi-class problems

> Example: Predicting if email is Spam (1) or Not-spam (0) Actual: y=1 (Spam), Predicted: y_hat=0.85 BCE Loss = -(1*log(0.85) + 0*log(0.15)) = -log(0.85) = 0.163 Small loss means prediction (0.85) is close to actual (1) — good! If predicted y_hat=0.1: BCE Loss = -log(0.1) = 2.30 (large loss — bad)

Answer

Q4. Briefly describe the Perceptron Algorithm.

The **Perceptron Learning Algorithm** is an iterative algorithm that adjusts weights to correctly classify linearly separable data. It is guaranteed to converge if the data is linearly separable.

Algorithm Steps:

- Step 1 — Initialize: Set all weights wi=0 and bias b=0. Choose learning rate n (typically 0.1 to 1.0).
- Step 2 — For each training example (xi, yi): Compute output: y_hat = step(W^T*xi + b)
- Step 3 — Compute error: error = yi - y_hat
- Step 4 — Update weights: wj = wj + n * error * xij for each weight j
- Step 5 — Update bias: b = b + n * error
- Step 6 — Repeat steps 2-5 for all training examples (one epoch)
- Step 7 — Repeat epochs until no misclassification or max epochs reached

```
Update Rule: w_new = w_old + n * (target - output) * x If correct: error=0, no update If wrong (predicted 0, actual 1): weights increase If wrong (predicted 1, actual 0): weights decrease
```

> Example — AND Gate (n=1): Training: [0,0]->0, [0,1]->0, [1,0]->0, [1,1]->1 Start: w1=0, w2=0, b=0 Epoch 1, Example (1,1)->1: output = step(0*1 + 0*1 + 0) = step(0) = 0 error = 1 - 0 = 1 w1 = 0 + 1*1*1 = 1, w2 = 1, b = 1 Continue until all examples classified correctly.

Unit 3 — SVM, Kernels, Bayesian Models, Overfitting, HMM

Support Vectors | Kernel Functions | Naive Bayes | Bayesian Networks | Bayes Theorem

Answer

Q1. What is a Support Vector? Explain how SVM works with suitable examples.

Support Vectors are the data points that lie closest to the decision boundary (hyperplane). They are the critical/border points that directly influence the position and orientation of the separating hyperplane. If you move any support vector, the boundary changes.

How SVM works — Step by Step:

- Goal: Find the hyperplane that maximally separates two classes with the widest possible margin.
- Hyperplane: A line (2D), plane (3D), or hyperplane (higher-D) defined by: w^T*x + b = 0
- Margin: The distance between the hyperplane and the nearest support vectors = 2/||w||
- SVM maximizes this margin — wider margin = better generalization to unseen data

```
SVM Optimization (Hard Margin): Minimize: (1/2)||w||^2 Subject to: yi(w^T*xi + b) >= 1 for all training examples i Margin width = 2 / ||w|| (maximized by minimizing ||w||)
```

Soft Margin SVM — for non-perfectly separable data:

```
Minimize: (1/2)||w||^2 + C * SUM(xi_i) C = penalty parameter Large C: fewer errors, smaller margin (more complex boundary) Small C: allows more errors, larger margin (simpler boundary)
```

> Example — Email Classification: Data: Spam emails (class +1) and Ham emails (class -1) Features: x1=word count, x2=exclamation marks count SVM finds the hyperplane that best separates these two classes Support vectors: the spam and ham emails closest to the boundary Visual intuition: - Many possible lines can separate the classes - SVM picks the one with MAXIMUM distance from both classes - This maximizes confidence in predictions for new emails

Answer

Q2. What are SVM Kernels? Explain Linear, Polynomial, RBF, and Sigmoid kernels.

When data is **not linearly separable** in the original space, the **Kernel Trick** maps data to a higher-dimensional space where it becomes linearly separable — without explicitly computing the transformation. The kernel function K(x,z) computes the dot product in the transformed space.

```
K(x, z) = phi(x)^T * phi(z) [phi maps to higher dimension]
```

(i) Linear Kernel:

```
K(x, z) = x^T * z = SUM(xi * zi)
```

- No transformation — original feature space is used
- Best for linearly separable data
- Fastest computation, fewest parameters
- Use case: Text classification (many sparse features)

(ii) Polynomial Kernel:

```
K(x, z) = (x^T*z + c)^d c = constant (typically 0 or 1) d = degree of polynomial (e.g. d=2 for quadratic, d=3 for cubic)
```

- Creates polynomial decision boundaries
- d=1: equivalent to linear kernel
- Use case: Image classification, NLP tasks

(iii) RBF (Radial Basis Function) / Gaussian Kernel:

```
K(x, z) = exp( -||x - z||^2 / (2 * sigma^2) ) OR equivalently: exp( -gamma * ||x - z||^2 ) gamma = 1/(2*sigma^2) — controls the width of the Gaussian
```

- Most widely used kernel — works well in most cases
- sigma large: smoother boundary (underfitting risk)
- sigma small: wiggly boundary (overfitting risk)
- K(x,z) = 1 when x=z (same point), -> 0 as points get further apart
- Use case: Classification with no prior knowledge of data distribution

(iv) Sigmoid Kernel:

```
K(x, z) = tanh( alpha * x^T*z + c ) alpha = slope, c = intercept
```

- Mimics a two-layer neural network
- Not always positive semi-definite (may not be a valid kernel for all parameters)
- Use case: Neural network-like behavior in SVM

Answer

Q3. What is the role of Bayesian models in Machine Learning?

Bayesian models apply probability theory to machine learning, treating unknown quantities as random variables with probability distributions rather than fixed values. They provide a principled framework for incorporating prior knowledge and quantifying uncertainty.

Key roles:

- **Uncertainty Quantification:** Instead of a single prediction, Bayesian models give a probability distribution over predictions — crucial in medical diagnosis, self-driving cars.
- **Prior Knowledge Incorporation:** Domain knowledge can be encoded as prior distributions P(h), improving performance with limited data.
- **Overfitting Prevention:** Bayesian approach naturally regularizes by averaging over many hypotheses rather than committing to one.
- **Model Comparison:** Bayes factor allows comparing different models objectively using evidence P(D).
- **Generative Modeling:** Bayesian models can generate new data samples by learning the underlying data distribution.

> Medical Diagnosis Example: Prior: P(Cancer) = 0.01 (1% prevalence in population) Likelihood: P(Test+|Cancer) = 0.90 (test sensitivity) Bayesian model updates belief after seeing test result: P(Cancer|Test+) = P(Test+|Cancer)*P(Cancer) / P(Test+) This prevents over-reliance on test results and incorporates prevalence — a pure frequentist approach might miss this.

Answer

Q4. Define Naive Bayes Classifier and describe the Naive Bayes Algorithm.

The **Naive Bayes Classifier** is a probabilistic classifier based on Bayes Theorem with the "naive" assumption that all features are conditionally independent given the class label. Despite this simplifying assumption (which is rarely true in reality), it performs surprisingly well in practice.

```
Bayes Theorem: P(C|X) = P(X|C) * P(C) / P(X) Naive Bayes: P(C|x1,x2,...,xn) ∝
P(C) * P(x1|C) * P(x2|C) * ... * P(xn|C) [assumes P(xi,xj|C) = P(xi|C)*P(xj|C)
for all i,j – independence assumption]
```

Naive Bayes Algorithm:

- Step 1 — Training Phase: Calculate prior probabilities P(C) for each class C = count(C)/total
- Step 2 — Calculate likelihoods P(xi|C) for each feature xi and class C from training data
- Step 3 — Apply Laplace Smoothing: P(xi=v|C) = (count(xi=v in C)+1) / (count(C)+k) where k=number of unique values. Prevents zero probabilities.
- Step 4 — Classification: For new instance x, compute score for each class C:

```
Score(C) = P(C) * PROD_i P(xi|C) Prediction = argmax_C Score(C)
```

> Example — Play Tennis Prediction: Training: 14 days, 9 YES, 5 NO P(YES)=9/14=0.64, P(NO)=5/14=0.36 P(Sunny|YES)=2/9, P(Sunny|NO)=3/5 [Laplace: (2+1)/(9+3)=0.25, (3+1)/(5+3)=0.50] P(Warm|YES)=4/9, P(Warm|NO)=2/5 New day: Sunny, Warm Score(YES) = 0.64 * 0.25 * 0.44 = 0.070 Score(NO) = 0.36 * 0.50 * 0.40 = 0.072 Decision: NO (do not play tennis) Applications: Spam filtering, sentiment analysis, medical diagnosis

---

Answer

Q5. What is Overfitting? Explain when generalization performance increases due to overfitting.

Overfitting occurs when a machine learning model learns the training data too well — including noise and random fluctuations — resulting in poor performance on new, unseen data. The model essentially memorizes the training data instead of learning general patterns.

> Visual analogy: Imagine connecting all training data points with a wiggly curve. The curve passes through every single training point perfectly (100% training accuracy) but performs terribly on new test points because the wiggles are just noise. A straight line (simpler model) might miss some training points but generalizes better.

Signs of Overfitting:

- Training accuracy is very high (95-100%) but test accuracy is much lower (60-70%)
- The gap between training loss and validation loss keeps increasing
- Model is overly complex relative to the amount of training data

Circumstances where generalization CAN improve despite apparent overfitting:

- **Double Descent Phenomenon:** In very large models (deep neural networks), as model complexity increases past a certain point, generalization error decreases again — even while training error remains near zero. This is observed in modern deep learning.
- **Interpolation Regime:** When a model is expressive enough to exactly fit training data AND the training data is representative (low noise), overfitting can actually generalize well.
- **Benign Overfitting:** With very high-dimensional data (many features, few samples), even models that perfectly fit training data may generalize well if data is in a low-dimensional manifold.

- **Ensemble Methods:** Individually overfitted models (e.g. deep trees in Random Forest) can combine to generalize well through averaging.

Prevention of harmful Overfitting: Regularization (L1/L2), Dropout, Early Stopping, More training data, Cross-validation

---

Answer

Q6. Write notes on (i) Hidden Markov Model (ii) Bayesian Network

(i) Hidden Markov Model (HMM):

An HMM is a statistical model for sequential/time-series data where the system has hidden (unobservable) states that generate observable outputs. The model assumes the Markov property: the next state depends only on the current state (not the entire history).

HMM Components:

- States S = {S1, S2, ..., Sn}: Hidden states (not directly observable)
- Observations O = {O1, O2, ..., Om}: Observable symbols/outputs
- Transition matrix A: A[i][j] = P(next state = Sj | current state = Si)
- Emission matrix B: B[i][k] = P(observation = Ok | state = Si)
- Initial probabilities pi: pi[i] = P(starting in state Si)

```
Forward Algorithm (Evaluation Problem): alpha(t, s) = P(o1,...,ot, St=s) alpha(1, s) = pi(s) * B(s, o1) alpha(t+1, s') = B(s', o_t+1) * SUM_s[ alpha(t,s) * A(s,s') ]
```

Three Classic HMM Problems:

- Evaluation: Given model, find P(observation sequence) — Forward Algorithm
- Decoding: Find most likely hidden state sequence — Viterbi Algorithm
- Learning: Estimate model parameters from observations — Baum-Welch (EM) Algorithm

> Applications of HMM: - Speech recognition: hidden states = phonemes, observations = audio features - POS tagging: hidden states = parts of speech, observations = words - DNA analysis: hidden states = gene regions, observations = nucleotides - Stock market: hidden states = market conditions, observations = prices

(ii) Bayesian Network:

A Bayesian Network (also called Belief Network) is a probabilistic graphical model represented as a Directed Acyclic Graph (DAG). Each node represents a random variable, and edges represent conditional dependencies between variables.

```
Joint Probability Factorization: P(X1, X2, ..., Xn) = PROD_i P(Xi | Parents(Xi))
This factorization assumes conditional independence given parents.
```

Key Properties:

- Each node has a Conditional Probability Table (CPT)
- Encodes conditional independence assumptions compactly
- Supports both forward (prediction) and backward (diagnosis) inference

Classic Wet Grass Example: Rain -> Sprinkler -> WetGrass <- Rain CPTs: P(Rain=T) = 0.2 P(Sprinkler=T|Rain=T) = 0.01, P(Sprinkler=T|Rain=F) = 0.40 P(WetGrass=T|Rain=T, Spr=T) = 0.99 P(WetGrass=T|Rain=F, Spr=T) = 0.90 P(WetGrass=T|Rain=T, Spr=F) = 0.80 P(WetGrass=T|Rain=F, Spr=F) = 0.00 Query: P(Rain=T|WetGrass=T) requires marginalizing over Sprinkler states. Applications: Medical diagnosis, fault diagnosis, spam filtering

---

Answer

Q7. (a) Discriminative vs Generative Learning (b) Bayes Theorem (c) Coin Problem

(a) Discriminative vs Generative Learning:

Aspect	Discriminative	Generative
Learn P(Y|X) directly	Learn P(X|Y) and P(Y)	
Decision boundary	Data distribution	
Less	More	
Cannot generate	Can generate new samples	
Higher	Lower (but more flexible)	
SVM, Logistic Regression, Neural Networks	Naive Bayes, HMM, Bayesian Networks	

(b) Bayes Theorem:

```
P(h | D) = [ P(D | h) * P(h) ] / P(D) P(h|D) = Posterior – probability of hypothesis h given data D P(D|h) = Likelihood – probability of data D given hypothesis h P(h) = Prior – initial probability of hypothesis h P(D) = Evidence – total probability of data (normalizing constant)
```

(c) Coin Problem — Solved:

Two hypotheses: h1 = fair coin (P(H)=0.5), h2 = biased coin (P(H)=0.6). Priors: P(h1)=0.75, P(h2)=0.25. Observation: 5 flips = HHTTH (3 Heads, 2 Tails).

Step 1 — Compute Likelihoods: P(D|h1) = P(3H,2T | fair) = C(5,3) * (0.5)^3 * (0.5)^2 = 10 * 0.125 * 0.25 = 10 * 0.03125 = 0.3125 P(D|h2) = P(3H,2T | biased) = C(5,3) * (0.6)^3 * (0.4)^2 = 10 * 0.216 * 0.16 = 10 * 0.03456 = 0.3456 Step 2 — Compute Evidence P(D): P(D) = P(D|h1)*P(h1) + P(D|h2)*P(h2) = 0.3125*0.75 + 0.3456*0.25 = 0.2344 + 0.0864 = 0.3208 Step 3 — Apply Bayes Theorem: P(h1|D) = P(D|h1)*P(h1) / P(D) = (0.3125*0.75) / 0.3208 = 0.2344 / 0.3208 = 0.731 P(h2|D) = P(D|h2)*P(h2) / P(D) = (0.3456*0.25) / 0.3208 = 0.0864 / 0.3208 = 0.269 CONCLUSION: P(h1|D) = 0.731 > P(h2|D) = 0.269 => The coin is most likely a FAIR COIN (h1) (The prior P(h1)=0.75 was dominant here; even with a slightly higher likelihood for h2, the strong prior keeps h1 as the MAP estimate)

Unit 4 — Neural Networks, CNN, Regularization, Linear Regression

Solved Linear Regression | Feed-forward NN | Backpropagation | LeNet-5 | Bagging | k-Fold CV

Problem

Q1. Linear Regression: Calculate regression of Patients Age vs Blood Pressure (Solved)

Given data for 10 patients:

Patient	1	2	3	4	5	6	7	8	9	10
Age (x)	35	40	38	44	67	64	59	69	25	50
BP (y)	112	128	130	138	158	162	140	175	125	142

SOLUTION — Linear Regression: y = w1*x + w0 n = 10 patients Step 1 — Compute summary values: Age values: 35, 40, 38, 44, 67, 64, 59, 69, 25, 50 BP values: 112, 128, 130, 138, 158, 162, 140, 175, 125, 142 Sum(x) = 35+40+38+44+67+64+59+69+25+50 = 491 Sum(y) = 112+128+130+138+158+162+140+175+125+142 = 1410 x_bar = Sum(x)/n = 491/10 = 49.1 y_bar = Sum(y)/n = 1410/10 = 141.0 Step 2 — Compute Sum(x^2) and Sum(x*y): x^2: 1225+1600+1444+1936+4489+4096+3481+4761+625+2500 = 26157 x*y: 35*112+40*128+38*130+44*138+67*158+64*162+59*140+69*175+25*125+50*142 = 3920+5120+4940+6072+10586+10368+8260+12075+3125+7100 = 71566 Step 3 — Compute slope w1: w1 = [Sum(xy) - n*x_bar*y_bar] / [Sum(x^2) - n*x_bar^2] = [71566 - 10*49.1*141.0] / [26157 - 10*49.1^2] = [71566 - 69231.0] / [26157 - 24108.1] = 2335 / 2048.9 = 1.1396 ≈ 1.14 Step 4 — Compute intercept w0: w0 = y_bar - w1 * x_bar = 141.0 - 1.14 * 49.1 = 141.0 - 55.97 = 85.03 ≈ 85.03 ANSWER: Regression Equation: BP = 1.14 * Age + 85.03 Verification (predictions): Age=35: BP = 1.14*35 + 85.03 = 39.9+85.03 = 124.93 (actual: 112) Age=67: BP = 1.14*67 + 85.03 = 76.38+85.03 = 161.41 (actual: 158) Age=25: BP = 1.14*25 + 85.03 = 28.5+85.03 = 113.53 (actual: 125) Interpretation: For every 1 year increase in age, Blood Pressure increases by approximately 1.14 mmHg.

Answer

Q2. Write notes on: (i) Feed-forward NN (ii) Backpropagation (iii) CNN (iv) LeNet-5 (v) Bagging

(i) Feed-forward Neural Networks:

In a feed-forward network, information flows in one direction only — from input layer through hidden layers to output layer. There are no cycles or loops. Each layer is fully connected to the next layer.

- Each neuron computes: output = f(W*input + b)
- Training uses backpropagation to adjust weights
- Universal approximator — can approximate any continuous function given enough neurons

(ii) Backpropagation:

Backpropagation is the algorithm for training MLPs. It computes gradients of the loss with respect to each weight using the chain rule, then updates weights using gradient descent.

```
Forward: y_hat = f(W2 * f(W1*X + b1) + b2) Loss: L = (1/2)(y - y_hat)^2 Output
delta: d_out = (y-y_hat) * f'(z_out) Hidden delta: d_h = (W2^T * d_out) * f'(z_h)
Update: W = W + n * delta * input^T
```

(iii) Convolutional Neural Networks (CNN):

CNNs are specialized for grid-like data (images). Key operations: Convolution (feature extraction), Pooling (spatial reduction), Fully Connected (classification).

- Convolution: Filter slides over input computing dot products -> feature map
- Max Pooling: Takes maximum in each region -> reduces spatial dimensions
- Parameter sharing: Same filter reused across all positions -> fewer parameters
- Translation invariance: Detects features regardless of location in image

(iv) LeNet-5 Architecture (LeCun, 1998):

> LeNet-5 for Handwritten Digit Recognition (MNIST): Input: 32x32 grayscale image C1: 6 filters, 5x5 conv -> output: 6 x 28 x 28 (feature maps) S2: Average pooling 2x2 -> output: 6 x 14 x 14 C3: 16 filters, 5x5 conv -> output: 16 x 10 x 10 S4: Average pooling 2x2 -> output: 16 x 5 x 5 C5: 120 filters, 5x5 -> output: 120 x 1 x 1 F6: Fully connected -> 84 neurons, tanh activation Output: 10 neurons (digits 0-9), Softmax activation Key insight: Alternating conv+pool layers extract increasingly abstract features (edges -> shapes -> digit parts -> digits).

(v) Bagging (Bootstrap Aggregating):

Bagging is an ensemble method that trains multiple models on different bootstrap samples (random sampling with replacement) of the training data and combines their predictions.

- Create B bootstrap samples: each of size n, sampled with replacement from training data
- Train one base model (e.g. decision tree) on each bootstrap sample
- Classification: Final prediction = majority vote of all B models
- Regression: Final prediction = average of all B model predictions
- Key benefit: Reduces variance without increasing bias
- Random Forest = Bagging + Decision Trees + Random feature selection at each split

---

Answer

Q3. Write notes on: (i) MLP (ii) Neural Network Architecture (iii) Backpropagation Algorithm

(i) Multilayer Perceptron (MLP):

MLP is a class of feedforward neural network with one or more hidden layers between input and output. It overcomes the limitations of the single-layer Perceptron.

- Key capability: Can learn non-linear decision boundaries
- Universal approximation theorem: An MLP with one hidden layer and sufficient neurons can approximate any continuous function to arbitrary precision
- Training: Backpropagation + Gradient Descent
- Applications: Classification, Regression, Function Approximation, Pattern Recognition

(ii) Neural Network Architecture:

A neural network architecture defines the structure: number of layers, neurons per layer, connections, and activation functions.

- Shallow networks: Input + 1 hidden layer + Output — fast, simple tasks
- Deep networks (DNN): Many hidden layers — complex hierarchical features
- Width: More neurons per layer = more capacity per layer
- Depth: More layers = more abstract representations

- Activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU, Softmax

(iii) Backpropagation Algorithm (Complete):

- Step 1: Initialize all weights randomly (small values)
- Step 2: For each training batch, perform Forward Pass — compute all layer activations
- Step 3: Compute Loss L between prediction y_hat and true label y
- Step 4: Backward Pass — compute dL/dW for each layer using chain rule
- Step 5: Update weights: W = W - learning_rate * dL/dW
- Step 6: Repeat for all batches until convergence

```
Chain Rule: dL/dW1 = dL/dy_hat * dy_hat/dz2 * dz2/da1 * da1/dW1 Each "link" in
the chain is a local gradient — computed and multiplied back.
```

Answer

Q4. Write notes on: (i) Regularization (ii) K-Fold Cross Validation

(i) Regularization:

Regularization adds a penalty term to the loss function to prevent overfitting by discouraging the model from learning large weights.

```
L1 Regularization (Lasso): Loss = MSE + lambda * SUM|wi| -> Creates sparse models
(some weights become exactly 0) -> Acts as feature selection L2 Regularization
(Ridge): Loss = MSE + lambda * SUM(wi^2) -> Shrinks all weights toward (but not
to) zero -> Handles correlated features better lambda: hyperparameter — controls
strength of regularization lambda=0: no regularization, lambda large:
underfitting
```

Other regularization techniques:

- Dropout: Randomly deactivate neurons during training — forces redundancy, prevents co-adaptation
- Early Stopping: Stop training when validation loss starts increasing
- Data Augmentation: Artificially expand training set (flipping, rotating images)

(ii) K-Fold Cross Validation:

K-Fold CV is a model evaluation technique that uses all data for both training and testing, giving a more reliable performance estimate than a single train/test split.

- Step 1: Randomly shuffle dataset and split into k equal-sized folds
- Step 2: For each fold i (i = 1 to k):
- Use fold i as the test set
- Train the model on the remaining k-1 folds
- Record performance metric (accuracy, F1, RMSE, etc.)
- Step 3: Final score = mean of k scores. Standard deviation shows consistency.

> Example: 5-Fold CV on 100 examples Fold size = 20 examples each Run 1: Train on folds 2,3,4,5 (80 ex) -> Test on fold 1 (20 ex) -> Acc: 84% Run 2: Train on folds 1,3,4,5 -> Test on fold 2 -> Acc: 87% Run 3: Train on folds 1,2,4,5 -> Test on fold 3 -> Acc: 83% Run 4: Train on folds 1,2,3,5 -> Test on fold 4 -> Acc: 86% Run 5: Train on folds 1,2,3,4 -> Test on fold 5 -> Acc: 85% Final accuracy = (84+87+83+86+85)/5 = 85% Common choices: k=5 or k=10. k=n is called Leave-One-Out CV (LOOCV).

Unit 5 — LDA, PCA, Clustering, Reinforcement Learning

LDA Projection (2 Solved Problems) | K-Means | Hierarchical Clustering | Q-Learning

Problem

Q1. LDA Projection — Class w1={(4,2),(2,4),(3,6),(4,4)} Class w2={(9,10),(6,8),(9,5),(8,7),(10,8)}

Find the Linear Discriminant projection that best separates the two classes.

SOLUTION Class w1: (4,2), (2,4), (3,6), (4,4) n1=4 Class w2: (9,10), (6,8), (9,5), (8,7), (10,8) n2=5 Step 1 — Compute Class Means: mu1 = (1/4)*[(4+2+3+4), (2+4+6+4)] = (1/4)*[13, 16] = [3.25, 4.00] mu2 = (1/5)*[(9+6+9+8+10), (10+8+5+7+8)] = (1/5)*[42, 38] = [8.40, 7.60] Step 2 — Compute Within-Class Scatter Matrices (Sw): For w1 — deviations from mu1=[3.25, 4.00]: (4,2): d=(0.75,-2.00) -> d*d^T = [[0.5625, -1.5],[-1.5, 4.0]] (2,4): d=(-1.25, 0.00) -> d*d^T = [[1.5625, 0.0],[0.0, 0.0]] (3,6): d=(-0.25, 2.00) -> d*d^T = [[0.0625, -0.5],[-0.5, 4.0]] (4,4): d=(0.75, 0.00) -> d*d^T = [[0.5625, 0.0],[0.0, 0.0]] S1 = sum = [[2.75, -2.0],[-2.0, 8.0]] For w2 — deviations from mu2=[8.40, 7.60]: (9,10): d=(0.60, 2.40) -> [[0.36, 1.44],[1.44, 5.76]] (6, 8): d=(-2.40, 0.40) -> [[5.76, -0.96],[-0.96, 0.16]] (9, 5): d=(0.60,-2.60) -> [[0.36, -1.56],[-1.56, 6.76]] (8, 7): d=(-0.40,-0.60) -> [[0.16, 0.24],[0.24, 0.36]] (10,8): d=(1.60, 0.40) -> [[2.56, 0.64],[0.64, 0.16]] S2 = sum = [[9.20, -0.20],[-0.20, 13.20]] Sw = S1 + S2 = [[11.95, -2.20],[-2.20, 21.20]] Step 3 — Compute Between-Class Scatter (Sb): (mu1-mu2) = [3.25-8.40, 4.00-7.60] = [-5.15, -3.60] Sb = (mu1-mu2)*(mu1-mu2)^T = [[-5.15],[-3.60]] * [[-5.15, -3.60]] = [[26.52, 18.54],[18.54, 12.96]] Step 4 — Compute optimal projection w = Sw^(-1) * (mu1 - mu2): Sw^(-1): det(Sw) = 11.95*21.20 - (-2.20)*(-2.20) = 253.34 - 4.84 = 248.50 Sw^(-1) = (1/248.50) * [[21.20, 2.20],[2.20, 11.95]] = [[0.08530, 0.00885],[0.00885, 0.04810]] w = Sw^(-1) * (mu1-mu2) = [[0.08530, 0.00885],[0.00885, 0.04810]] * [-5.15, -3.60] = [0.08530*(-5.15)+0.00885*(-3.60), 0.00885*(-5.15)+0.04810*(-3.60)] = [-0.4393-0.03186, -0.04558-0.17316] = [-0.4712, -0.2187] ANSWER: Optimal LDA projection direction: w = [-0.471, -0.219] (Normalize: ||w|| = sqrt(0.471^2 + 0.219^2) = 0.520) w_normalized = [-0.906, -0.421] Projects 1D scores: z = w^T * x Class w1 centre: w^T*mu1 = -0.906*3.25 + (-0.421)*4.00 = -2.945 - 1.684 = -4.63 Class w2 centre: w^T*mu2 = -0.906*8.40 + (-0.421)*7.60 = -7.610 - 3.200 = -10.81 Classes are well separated in this 1D projection!

Problem

Q2. LDA Projection — Class w1={(1,2),(2,3),(3,3),(4,5),(5,5)}, w2={(4,2),(5,0),(5,2),(3,2),(5,3),(6,3)}

SOLUTION Class w1: (1,2),(2,3),(3,3),(4,5),(5,5) n1=5 Class w2: (4,2),(5,0),(5,2),(3,2),(5,3),(6,3) n2=6 Step 1 — Class Means: mu1 = (1/5)*[(1+2+3+4+5), (2+3+3+5+5)] = (1/5)*[15, 18] = [3.00, 3.60] mu2 = (1/6)*[(4+5+5+3+5+6), (2+0+2+2+3+3)] = (1/6)*[28, 12] = [4.667, 2.00] Step 2 — Within-Class Scatter: For w1 (mu1=[3.00, 3.60]): (1,2): d=[-2,-1.6] -> [[4.0, 3.2],[3.2, 2.56]] (2,3): d=[-1,-0.6] -> [[1.0, 0.6],[0.6, 0.36]] (3,3): d=[0,-0.6] -> [[0.0, 0.0],[0.0, 0.36]] (4,5): d=[1, 1.4] -> [[1.0, 1.4],[1.4, 1.96]] (5,5): d=[2, 1.4] -> [[4.0, 2.8],[2.8, 1.96]] S1 = [[10.0, 8.0],[8.0, 7.20]] For w2 (mu2=[4.667, 2.00]): (4,2): d=[-0.667, 0.0] -> [[0.444, 0.000],[0.000, 0.000]] (5,0): d=[0.333,-2.0] -> [[0.111,-0.667],[-0.667, 4.000]] (5,2): d=[0.333, 0.0] -> [[0.111, 0.000],[0.000, 0.000]] (3,2): d=[-1.667, 0.0] -> [[2.778, 0.000],[0.000, 0.000]] (5,3): d=[0.333, 1.0] -> [[0.111, 0.333],[0.333, 1.000]] (6,3): d=[1.333, 1.0] -> [[1.778, 1.333],[1.333, 1.000]] S2 = [[5.333, 1.000],[1.000, 6.000]] Sw = S1 + S2 = [[15.333, 9.000],[9.000, 13.200]] Step 3 — Between-Class Scatter: (mu1-mu2) = [3.00-4.667, 3.60-2.00] = [-1.667, 1.60] Sb = (mu1-mu2)(mu1-mu2)^T = [[2.779, -2.667],[-2.667, 2.560]] Step 4 — Projection w = Sw^(-1) * (mu1-mu2): det(Sw) = 15.333*13.200 - 9.0*9.0 = 202.4 - 81.0 = 121.4 Sw^(-1) = (1/121.4)*[[13.200,-9.000],[-9.000,15.333]] = [[0.1087,-0.0741],[-0.0741, 0.1263]] w = Sw^(-1) * [-1.667, 1.60] = [0.1087*(-1.667)+(-0.0741)*(1.60), (-0.0741)*(-1.667)+(0.1263)*(1.60)] = [-0.1812-0.1186, 0.1235+0.2021] = [-0.2998, 0.3256] ANSWER: w = [-0.300, 0.326] (LDA projection direction) Projected class means: w^T*mu1 = -0.300*3.00 + 0.326*3.60 = -0.90+1.174 = +0.274 w^T*mu2 = -0.300*4.667 + 0.326*2.00 = -1.40+0.652 = -0.748 Decision boundary (midpoint): (0.274-0.748)/2 = -0.237 Classification rule: If w^T*x > -0.237 -> Class w1, else -> Class w2

Answer

Q3. Describe the steps to calculate a lower-dimensional subspace using LDA.

Linear Discriminant Analysis (LDA) finds the projection directions that maximize the ratio of between-class scatter to within-class scatter, creating the most discriminative lower-dimensional representation.

Complete LDA Steps:

- Step 1 — Compute class mean vectors mu_k for each class k: mu_k = (1/n_k) * SUM_{xi in class k} xi
- Step 2 — Compute Within-Class Scatter Matrix Sw: Sw = SUM_k SUM_{xi in k} (xi - mu_k)(xi - mu_k)^T
- Step 3 — Compute Between-Class Scatter Matrix Sb: Sb = SUM_k n_k * (mu_k - mu)(mu_k - mu)^T where mu = overall mean of all data
- Step 4 — Solve the Generalized Eigenvalue Problem: Sb * w = lambda * Sw * w OR equivalently find eigenvectors of Sw^(-1) * Sb
- Step 5 — Sort eigenvectors by eigenvalue (descending). Each eigenvector = one discriminant direction.
- Step 6 — Select top d eigenvectors (d < number of classes - 1) to form projection matrix W.
- Step 7 — Project data: Z = W^T * X (reduced to d-dimensional space)

```
LDA Objective: maximize J(w) = (w^T Sb w) / (w^T Sw w) Solution: w* = Sw^(-1) * (mu1 - mu2) [for 2-class case] Max discriminant directions = min(num_features, num_classes - 1)
```

Answer

Q4. What is Reinforcement Learning? Elements, Scope, and Limitations.

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make sequential decisions by interacting with an environment. The agent receives rewards (positive) or penalties (negative) based on its actions, and its goal is to maximize cumulative long-term reward.

> Analogy: Training a dog Agent = dog, Environment = room, Actions = sit/stand/roll over Reward = treat (positive), Penalty = scolding (negative) Over time the dog learns which actions get treats (maximizes reward)

Key Elements of Reinforcement Learning:

- **Agent:** The learner/decision-maker that takes actions (e.g. robot, game player)
- **Environment:** Everything the agent interacts with (e.g. game world, physical world)
- **State (S):** Current situation/configuration of the environment
- **Action (A):** Set of all possible moves the agent can make
- **Reward (R):** Scalar feedback signal indicating goodness of action in a state
- **Policy (pi):** Strategy that maps states to actions — what the agent is learning
- **Value Function V(s):** Expected cumulative reward starting from state s
- **Q-Function Q(s,a):** Expected reward for taking action a in state s
- **Discount factor (gamma):** 0<=gamma<=1 — how much future rewards are valued vs immediate

```
Q-Learning Update Rule: Q(s,a) <- Q(s,a) + alpha * [r + gamma*max_a' Q(s',a') - Q(s,a)] alpha=learning rate, r=reward, s'=next state, gamma=discount factor
```

Scope / Applications of RL:

- Game playing: AlphaGo (chess, Go), Atari games (DQN)
- Robotics: Robot arm manipulation, locomotion, navigation
- Recommendation systems: Personalized content delivery

- Autonomous vehicles: Self-driving car decision making
- Finance: Algorithmic trading, portfolio management
- Healthcare: Drug dosage optimization, treatment planning

Limitations of RL:

- Sample inefficiency: Needs millions of interactions to learn (expensive in real world)
- Reward design (reward hacking): Poorly designed rewards lead to unintended behaviors
- Exploration vs exploitation dilemma: Hard to balance trying new actions vs using known good ones
- Non-stationary environments: If environment changes, learned policy may become obsolete
- Partial observability: Agent may not have full state information
- Safety concerns: Agent may take harmful actions during exploration phase

---

Answer

Q5. Write short notes on: (i) LDA (ii) PCA (iii) K-Means (iv) Hierarchical Clustering (v) Clustering

(i) LDA — Linear Discriminant Analysis:

Supervised dimensionality reduction technique that finds projection directions maximizing class separability (maximizes between-class variance, minimizes within-class variance). Maximum d = min(features, classes-1).

```
J(w) = (w^T Sb w) / (w^T Sw w) -> maximize this ratio
```

(ii) PCA — Principal Component Analysis:

Unsupervised dimensionality reduction technique that finds orthogonal directions (principal components) of maximum variance in the data. Ignores class labels.

- Step 1: Standardize data (zero mean, unit variance)
- Step 2: Compute covariance matrix C = X^T*X / n
- Step 3: Find eigenvalues and eigenvectors of C
- Step 4: Sort by eigenvalue (descending) -> principal components
- Step 5: Project data onto top k eigenvectors: Z = X*W

> PCA vs LDA: PCA: Unsupervised, maximizes variance, good for compression/visualization LDA: Supervised, maximizes class separation, better for classification Use PCA when labels are unavailable; LDA when labels exist and classification is the goal.

(iii) K-Means Algorithm:

Partitional clustering algorithm that divides n data points into k clusters by minimizing within-cluster sum of squared distances.

- Step 1: Initialize k centroids randomly (K-Means++ for better init)
- Step 2: Assign each point to nearest centroid: c(i) = argmin_k ||xi - mu_k||^2
- Step 3: Update centroids: mu_k = mean of all points in cluster k
- Step 4: Repeat steps 2-3 until centroids do not change
- Time complexity: O(n*k*d*iterations), where d=dimensions
- Limitations: Must specify k, sensitive to outliers and initialization, assumes spherical clusters

(iv) Hierarchical Clustering:

Builds a hierarchy of clusters represented as a dendrogram (tree diagram). Does not require specifying k in advance.

- **Agglomerative (Bottom-Up — more common):**
- Start: Each point is its own cluster (n clusters)
- Merge the two closest clusters at each step
- Continue until all points are in one cluster
- **Divisive (Top-Down):**
- Start: All points in one cluster
- Recursively split the cluster until each point is its own cluster
- **Linkage criteria:** Single linkage (min dist), Complete linkage (max dist), Average linkage, Ward (minimize variance)

(v) Clustering — Overview:

Clustering is an unsupervised learning task of grouping data points such that points in the same group (cluster) are more similar to each other than to those in other groups. There is no predefined class label.

Types of clustering:

- Partitional: Divides data into k non-overlapping groups (K-Means, K-Medoids)
- Hierarchical: Builds a tree of clusters (Agglomerative, Divisive)
- Density-based: Finds arbitrarily shaped clusters (DBSCAN, OPTICS)
- Model-based: Fits statistical models to data (Gaussian Mixture Models)

Similarity measures:

```
Euclidean distance: d(x,y) = sqrt(SUM(xi-yi)^2) Manhattan distance: d(x,y) =
SUM|xi-yi| Cosine similarity: cos(x,y) = (x.y)/(||x||*||y||)
```

Evaluation metrics:

- Silhouette score: measures how similar a point is to its cluster vs other clusters
- Davies-Bouldin index: ratio of within-cluster to between-cluster distances (lower = better)
- Elbow method: plot WCSS vs k, choose k at the "elbow" point

> Real-world Applications of Clustering: - Customer segmentation (marketing — group customers by buying behavior) - Document clustering (group similar news articles) - Image segmentation (group pixels by color/texture) - Anomaly detection (points far from any cluster = outliers) - Gene expression analysis (group genes with similar expression patterns)

Unit — Quick Formula Reference Sheet

All important formulas from Units 2–5 at a glance

Formula / Concept	Expression
Perceptron Output	`y = step(W^T*X + b)`
Perceptron Update	`wi = wi + n*(target-output)*xi`
Sigmoid	`sigma(x) = 1/(1+e^-x); sigma'(x) = sigma(x)*(1-sigma(x))`
ReLU	`ReLU(x) = max(0, x)`
Backprop — output delta	`d_out = (y-y_hat) * sigma'(z_out)`
Backprop — weight update	`W = W + n * delta * input^T`
MSE Loss	`MSE = (1/n)*SUM(yi - y_hat_i)^2`
Cross-Entropy Loss	`CE = -(1/n)*SUM[yi*log(y_hat_i) + (1-yi)*log(1-y_hat_i)]`
Bayes Theorem	`P(h|D) = P(D|h)*P(h) / P(D)`
Naive Bayes	`P(C|x1..xn) ∝ P(C) * PROD P(xi|C)`
SVM Hard Margin	`Minimize (1/2)||w||^2 s.t. yi(w^T xi+b) >= 1`
SVM Margin Width	`Margin = 2 / ||w||`
RBF Kernel	`K(x,z) = exp(-gamma*||x-z||^2)`
Linear Regression w1	`w1 = [SUM(xy) - n*xbar*ybar] / [SUM(x^2) - n*xbar^2]`
Linear Regression w0	`w0 = ybar - w1*xbar`
L1 Regularization	`Loss = MSE + lambda*SUM|wi|`
L2 Regularization	`Loss = MSE + lambda*SUM(wi^2)`
LDA Objective	`J(w) = (w^T Sb w)/(w^T Sw w); w* = Sw^(-1)*(mu1-mu2)`
Within-class Scatter	`Sw = SUM_k SUM_{xi in k} (xi-mu_k)(xi-mu_k)^T`
Between-class Scatter	`Sb = SUM_k n_k*(mu_k-mu)(mu_k-mu)^T`
K-Means Objective	`Minimize SUM_k SUM_{xi in Ck} ||xi-mu_k||^2`
Euclidean Distance	`d(x,y) = sqrt(SUM(xi-yi)^2)`
Q-Learning Update	`Q(s,a) <- Q(s,a)+alpha*[r+gamma*max Q(s',a')-Q(s,a)]`
PCA: Variance %	`Var% for PC_i = lambda_i / SUM(all lambda_j) * 100`